

Recap

- Data types:

`int` (45,-3,6) `float` (0.134,5.12) `char` ('A','c','+', '@')

- ASCII code:

`'A'`(65) `'0'`(48) `'a'`(97)

- Array:

`int` a[5] `char` name[] = "Eddie"

- Basic I/O in C: `scanf()` `printf()`
- Basic I/O in C++: `std::cin` `std::cout`



Xi'an Jiaotong-Liverpool University

西交利物浦大學

MFE113TC

Programming Foundation for Intelligent Manufacturing Engineering

Lecture 03 Operators in C/C++

Dr. Jiachen Liu

jiachen.liu@xjtlu.edu.cn

Fall, 2026

Outline

- Basic Operations and Expressions
 - Basic Operations
 - Assignment Operator
 - Unary operators
 - Relational Operations
 - If ... else ...
 - Logical Operations
- Data Types with Basic Operators
- Type Conversions
- What is function?

Basic Operations and Expressions

Basic Operations

- C provides operators for all basic mathematical functions using the standard symbols:
 - Multiplication: $*$
 - Division: $/$
 - Subtraction: $-$
 - Addition: $+$
- Constants and variables can be operated on these symbols;

Basic Operations and Expressions

Basic Operations

- Example:

```
#include <stdio.h>
int main()
{
    int a=3+5;
    printf("a's value is: %d\n", a);

    a = a+3;
    printf("a's value is: %d\n", a);

    const int b=7;
    a = b+2;
    printf("a's value is: %d\n", a);

    b = a-1;
    return 0;
}
```

Constant

Can not be
modified

Basic Operations and Expressions

Basic Operations

- There are also modulo operators (%). It returns the remainder of a division.
 - E.g., $5/3=1$, and the remainder is 2;
 - $5/3$ would give the value of 1;
 - $5\%3$ would give the value of 2.

Example:

$$7/3 = 2 \dots\dots\dots 1$$

$$16/5 = 3 \dots\dots\dots 1$$

$$15/5 = 3 \dots\dots\dots 0$$



$$7\%3 = 1$$

$$16\%3 = 1$$

$$15\%3 = 0$$

Basic Operations and Expressions

Bindings / Precedence

- As in mathematics, operations are equipped with different binding strengths:
 - E.g., multiplication (*) is stronger than addition (+).
- Please use **parenthesis ()** to ensure desired results.

```
#include <stdio.h>
int main()
{
    int x=3, y=5, w, z;
    w=x+y*5;
    z=(x+y)*5;
    printf(" w equals to %d\n z equals to %d\n", w, z);
    return 0;
}
```

Basic Operations and Expressions

Assignment Operator

- Assignments can be rewritten in a more shorthand way:

$i = i + 2$ is the same as $i += 2$

$i = i - 7$ is the same as $i -= 7$

$i = i * 3$ is the same as $i *= 3$

$i = i / 4$ is the same as $i /= 4$

- The assignment shorthand is denoted: **var op= value.**

Remember it

Basic Operations and Expressions

Unary operators (including one variable)

- The first assigns positive 3 to a;
- The second assigns the negative of a to b;

C Operations	Operators	Examples
Positive	+	$a = +3$
Negative	-	$b = -a$
Logic 'Not'	!	$!c$

If c is true, !c is false
If c is false, !c is true

Basic Operations and Expressions

Unary operators (including one variable)

- Example:

```
#include <stdio.h>
int main()
{
    int a = 5;
    int b = -3;

    // Unary operators
    int positiveA = +a; // 5, unary plus doesn't change the value)
    int negativeB = -b; // 3, unary minus changes the sign

    // Arithmetic operators
    int sum = a + b; // sum will be 2 (addition)
    int difference = a - b; // difference will be 8 (subtraction)
    return 0;
}
```

Basic Operations and Expressions

Unary operators (including one variable)

- $i++$ is equivalent to $i = i + 1$;
- $i--$ is equivalent to $i = i - 1$.

C Operations	Operators	Examples
Increment	<code>++</code>	<code>i++</code> or <code>++i</code>
Decrement	<code>--</code>	<code>i--</code> or <code>--i</code>

Basic Operations and Expressions

Pre / Post Increment / Decrement

- $++i$ and $i++$, or $--i$ and $i--$;
- There are different, the location of the operator ($++$ or $--$) decides when the operation is implemented;
- Operation before the variable ($++i$ and $--i$) is a pre operation;

Basic Operations and Expressions

Pre / Post Increment / Decrement

- Operation after the variable (+ + i and - - i) is a pre operation;

```
#include <stdio.h>
int main()
{
    int a=9;
    printf("++a is equal to %d\n", ++a);
    printf("a is equal to %d\n", a);
    return 0;
}
```



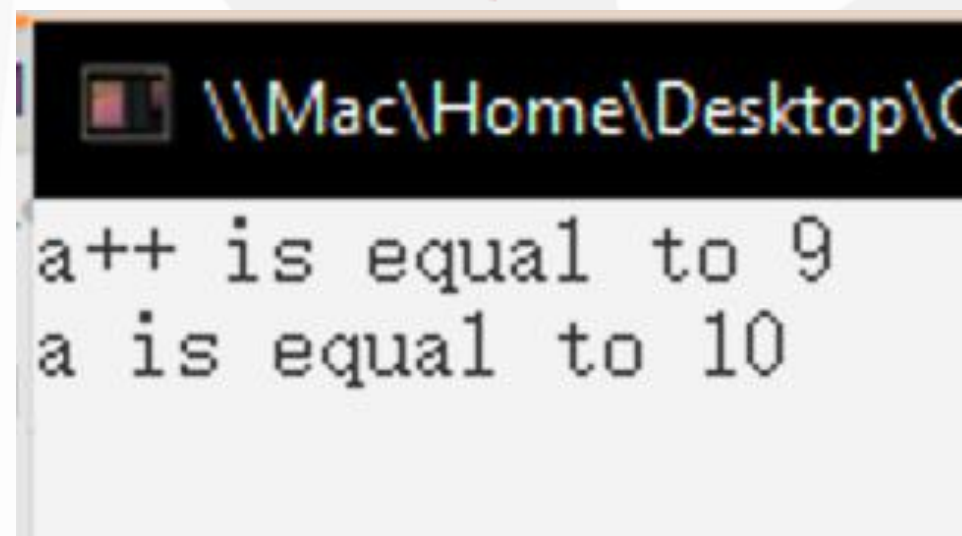
```
\\Mac\Home\Desktop\Conso
++a is equal to 10
a is equal to 10
```

Basic Operations and Expressions

Pre / Post Increment / Decrement

- Operation after the variable (i+ + and i - -) is a post operation;

```
#include <stdio.h>
int main()
{
    int a=9;
    printf("a++ is equal to %d\n", a++);
    printf("a is equal to %d\n", a);
    return 0;
}
```



```
\\Mac\Home\Desktop\C
a++ is equal to 9
a is equal to 10
```

Basic Operations and Expressions

Pre / Post Increment / Decrement

- Comparison between a++ and ++a;

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a=9;
```

```
    printf("++a is equal to %d\n", ++a);
```

```
    printf("a is equal to %d\n", a);
```

```
    printf("a++ is equal to %d\n", a++);
```

```
    printf("a is equal to %d\n", a);
```

```
    return 0;
```

```
}
```

Step 1: a = a+1;

Step 2: printf("%d", a);

Step 1: printf("%d", a);

Step 2: a = a+1;

++a is equal to 10
a is equal to 10
a++ is equal to 10
a is equal to 11

Basic Operations and Expressions

Relational Operations

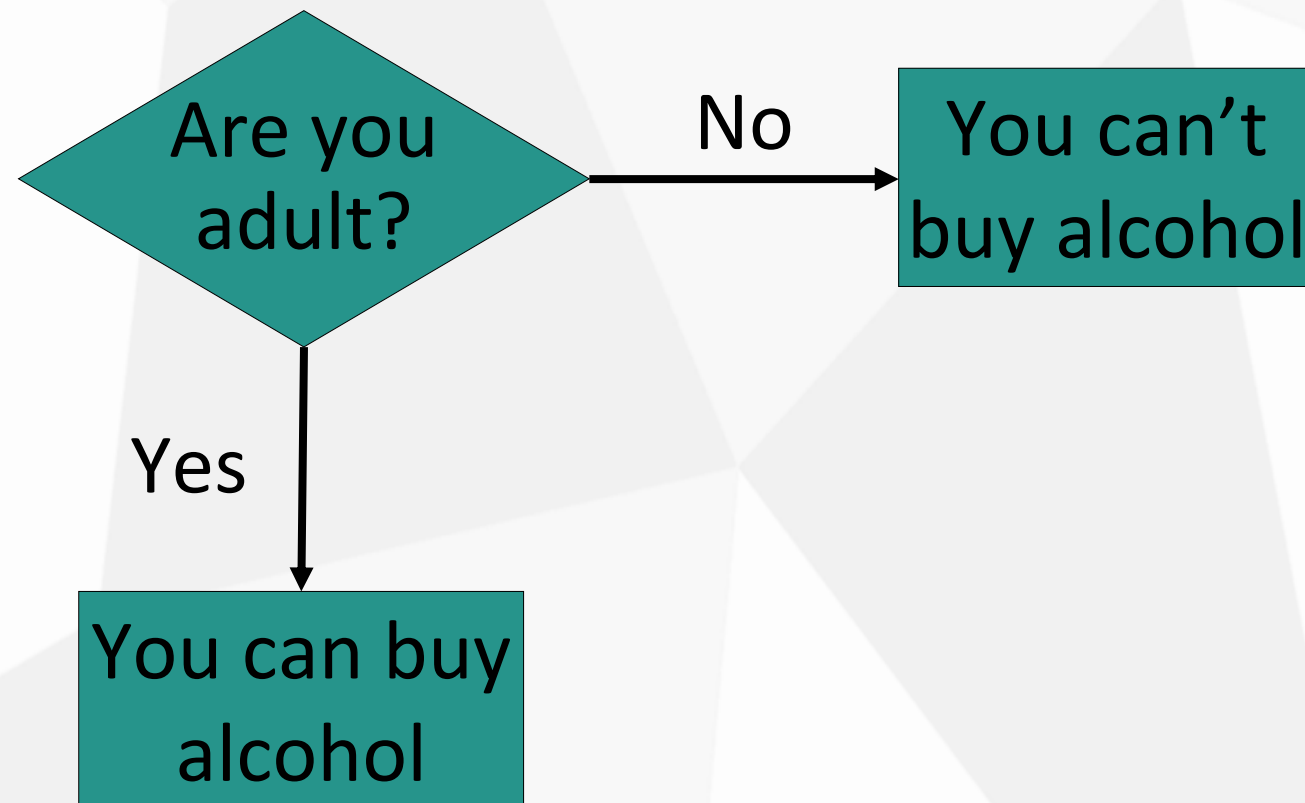
- The relational operations are used to compare expressions.

Operator	Meaning
<	is less than
<=	is less than or equal to
!=	is not equal
==	is equal to
>	is greater than
>=	is greater than or equal to

Only **==** is a relational Operation,
= is an assignment operation.
They are totally different

Basic Operations and Expressions

Relational Operations & **if ... else ...**



Basic Operations and Expressions

Relational Operations & **if ... else ...**

Operator	Meaning
<	is less than
<=	is less than or equal to
!=	is not equal
==	is equal to
>	is greater than
>=	is greater than or equal to

```
#include <stdio.h>

#define LEAGLAGE 18

int main()
{
    int your_age;
    scanf("%d", &your_age);
    if (your_age >= LEAGLAGE) {
        printf("You can buy alcohol\n");
    }
    else {
        printf("You can't buy alcohol\n");
    }
    return 0;
}
```

Basic Operations and Expressions

Relational Operations & **if ... else ...**



```
#include <stdio.h>

int main()
{
    int a = 0;

    if (a = 1) {
        printf("I will definitely hurt people\n");
    }
    else{
        printf("I will never hurt people\n");
    }
    return 0;
}
```

Anything wrong?

Basic Operations and Expressions

Relational Operations & **if ... else ...**



```
#include <stdio.h>

int main()
{
    int a = 0;

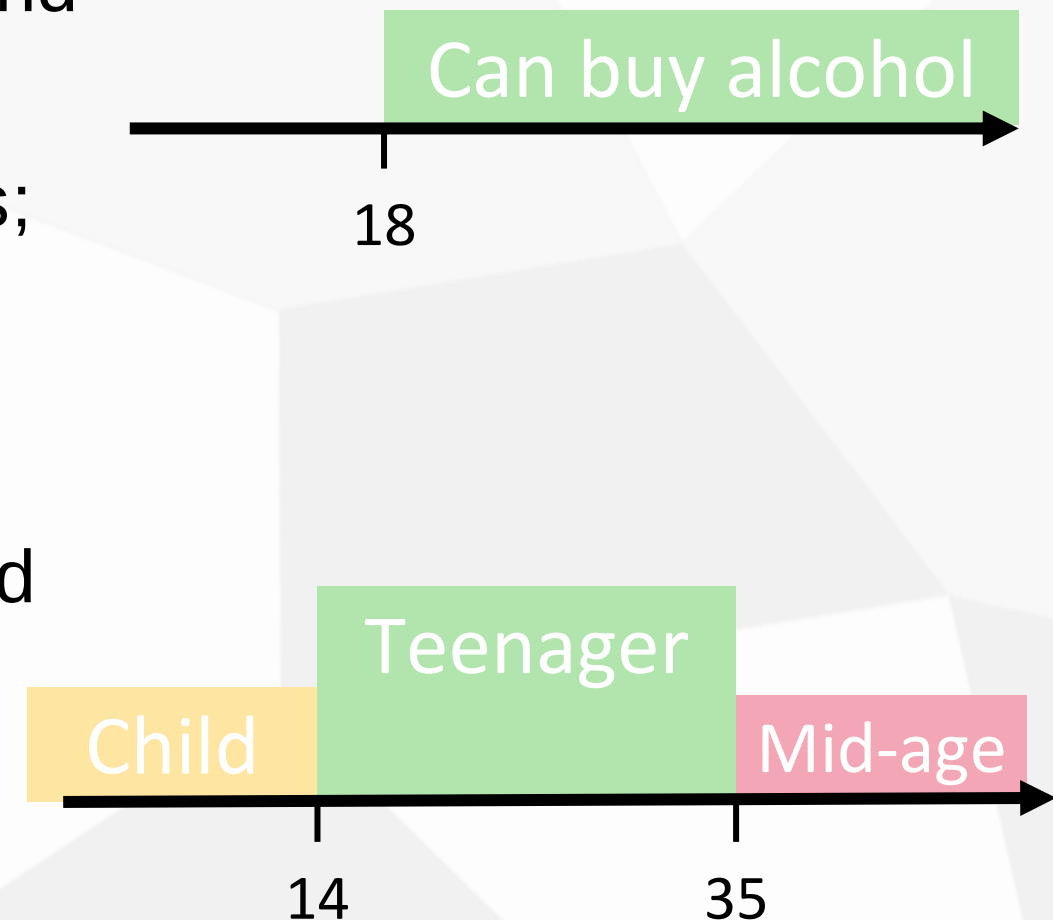
    if (a = 1){
        printf("I will definitely hurt people\n");
    }
    else{
        printf("I will never hurt people\n");
    }
    return 0;
}
```

Remember: '==' is the relational OP, '=' is the assignment OP !!!

Basic Operations and Expressions

Logical Operations

- The combination of **relational operators** and **'if...else...'** statements helps us make choices in situations with two distinct states;
- However, there are many situations with complex definitions that cannot be classified based on a single value.
- For example, teenagers are defined as individuals whose age falls between 14 and 35.



Basic Operations and Expressions

Logical Operations

```
#include <stdio.h>
int main()
{
    int number;
    printf("Please input your age:\n");
    scanf("%d", &number);
    if ((number>14) && (number<=35))
        printf("You are a teenager!\n");
    else
        printf("You are not a teenager!\n");
    return 0;
}
```

Complex text expressions:
Verify whether the number
lies within a range

Higher Precedence

Operator	Meaning
&&	Logical “and”: True if both arguments are True
	Logical “or”: True if one or both argument are True
!	Logical “Not”: reverse the argument

Still an ‘Unary OP’

Basic Operations and Expressions

Logical Operations

Operator	Meaning
&&	Logical “and”: True if both arguments are True
	Logical “or”: True if one or both argument are True
!	Logical “Not”: reverse the argument

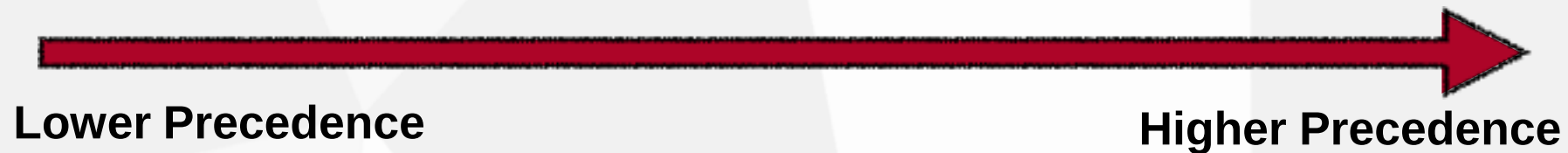
Still an ‘Unary OP’

Op1	Op2	Op1 && Op2	Op1 Op2	!Op1
1	1	1	1	0
0	1	0	1	1
0	0	0	0	1
1	0	0	1	0

Basic Operations and Expressions

Operators with Precedence

Assignment Operators	Relational Operators	Arithmetic Operators
=, +=, -=, *=, /=, %=, etc.	==, >=, >, <=, <, !=, etc.	+, -, *, /, %, etc.



$x > y + 2$ represents the same as $x > (y + 2)$

$x = y > 2$ represents the same as $x = (y > 2)$

Basic Operations and Expressions

Logical Operators with Precedence

- If $a = 1$, $b = 0$, and $c = 0$. What's the result of ' $a \parallel b \&\& c$ ' ?

Basic Operations and Expressions

Logical Operators with Precedence

- If $a = 1$, $b = 0$, and $c = 0$. What's the result of ' $a \parallel b \&\& c$ ' ?

$a \parallel b \&\& c$



$1 \parallel 0 = 1$



1

Basic Operations and Expressions

Logical Operators with Precedence

```
#include <stdio.h>

int main()
{
    int a = 1, b = 0, c = 0;

    int result1 = a || b && c;
    printf("Result1 (a && b || c): %d\n", result1);

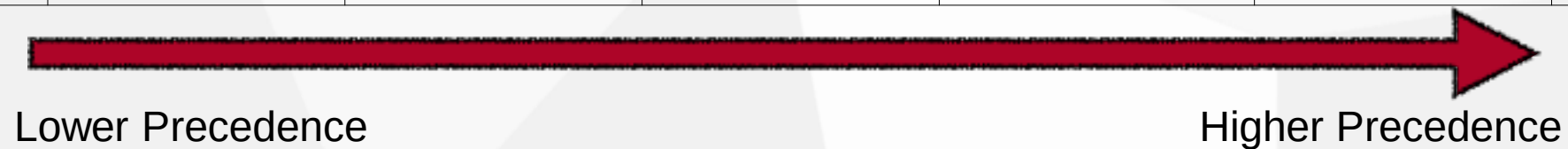
    return 0;
}
```

- The priority of '&&' is **higher than** that of '||'.

Basic Operations and Expressions

Logical Operators with Precedence

Assignment Operators		&&	Relational Operators	Arithmetic Operators	Unary Operators +, -, !, ++, --	()
----------------------	--	----	----------------------	----------------------	------------------------------------	----



`x=3>5 && 10>23 || 4>2`

represents the same as

`x=((3>5) && (10>23)) || (4>2)`

which means

`x=((False && False) || True)`

Basic Operations and Expressions

General Priorities of Operators

- Parenthesis **(Highest)**: ()
- Unary Operators: + +, - -, !, etc.;
- Arithmetic Operators: -, +, *, /;
- Relational Operators: <, >, <=, >=;
- Logical Operators: ||, &&;
- Assignment Operators **(Lowest)**: =, +=, *=, etc.
- Again be noted: **always use ()** to ensure your expected operation sequence.

Basic Operations and Expressions

Quiz

- Which of the following evaluates to true?
 - Assuming $P = \text{true}$, $Q = \text{false}$, $R = \text{true}$
 - 1). $(!P \parallel R) \&\& Q$
 - 2). $!(Q \&\& R) \&\& P$
 - 3). $!(P \&\& R) \parallel Q$
 - 4). $P \&\& !Q \&\& !R$

Basic Operations and Expressions

Quiz

- Which of the following evaluates to true?
 - Assuming $P = \text{true}$, $Q = \text{false}$, $R = \text{true}$
 - 1). $(!P \parallel R) \&\& Q$ **false**
 - 2). $!(Q \&\& R) \&\& P$ **true**
 - 3). $!(P \&\& R) \parallel Q$ **false**
 - 4). $P \&\& !Q \&\& !R$ **false**

Break

Basic Operations and Expressions

Data Types with Basic Operators

Data type	Example	Bytes
int	35, -5, 68	2 or 4
float	0.123456	4
double	0.123456789012345	8
char	'0'(48), 'A'(65)	1

What will happen if we perform computations involving **different data types**?

The data with the lower type will normally be converted to the higher type automatically before computation. This is called type conversion.

Basic Operations and Expressions

Data Types with Basic Operators

- Real and integer numbers are stored differently;
- How does the computer deal with this?

```
#include <stdio.h>
int main()
{
    float answer;
    int x=24, y=10;
    answer=x/y;
    printf("The answer is: %f\n", answer);
    return 0;
}
```

-> The answer is **2.0**

Basic Operations and Expressions

Data Types with Basic Operators

- Why the answer is 2.0?

```
#include <stdio.h>
int main()
{
    float answer;
    int x=24, y=10;
    answer=x/y;
    printf("The answer is: %f\n", answer);
    return 0;
}
```

Step 1 Expression: $\text{int}(24/10) = \text{int}(2.4) = 2$

Step 2 Assignment: $\text{float}(2) = 2.0$

Basic Operations and Expressions

Type Conversions

- Why we need store data in different formats?
 - Data can be stored in the most compact form and converted when needed;
 - Disadvantage: Type mismatches can be missed by the compiler, it might possibly lead to losing information.
 - E.g., $x/y=24/10=2$.

Basic Operations and Expressions

Data Types with Basic Operators

- Real and integer numbers are stored differently;
- How does the computer deal with this?

```
#include <stdio.h>
int main()
{
    float answer;
    float x=24.0;
    int y=10;
    answer=x/y;
    printf("The answer is: %f\n", answer);
    return 0;
}
```

-> What is the answer?

The answer is: 2.400000

Basic Operations and Expressions

Data Types with Basic Operators

- Real and integer numbers are stored differently;
- How does the computer deal with this?

```
#include <stdio.h>
int main()
{
    float answer;
    int x=24;
    float y=10.0;
    answer=x/y;
    printf("The answer is: %f\n", answer);
    return 0;
}
```

-> What is the answer?

The answer is: 2.400000

Basic Operations and Expressions

Type Conversions

- -> The answer is 2.4, but why?
 - It refers to the **changing of one data type to another type**;
 - Two types of conversions:
Implicit; Explicit.

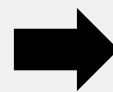
Basic Operations and Expressions

Implicit Conversions

- When data types are **mixed**, then the **type conversion** will be performed automatically;
- Remember that a **char is just a small integer value** and can be used in mathematical operations;

Refer to ASCII's encoding number

```
char c = 'A';  
printf("%d\n", c + 1);  
printf("%c\n", c + 1);
```



65 + 1 = 66
B

- There are some rules for these conversions.

Basic Operations and Expressions

Implicit Conversions

- Expression first, then assignment.

```
float answer;  
int x=24, y=10;  
answer=x/y;
```

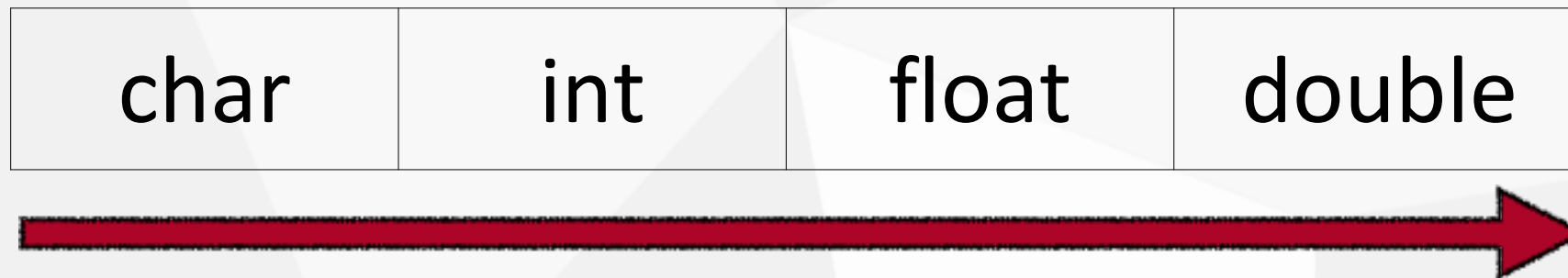
Expression first, because x and y are both int, so the result is **int(2.4) = 2**

Assignment, because answer is a **float**, so 2 is converted to float, **2.000000**

Basic Operations and Expressions

Implicit Conversions

- Commonly-used data type conversions.



e.g.,

char + int $\longrightarrow$ int + int $\longrightarrow$ int

Int/float $\longrightarrow$ float / float $\longrightarrow$ float

float - double $\longrightarrow$ double - double $\longrightarrow$ double

int + float + double $\longrightarrow$ double + double + double

Basic Operations and Expressions

Implicit Conversions

- **int** k=5, m=4, n; **float** x=1.5, y=2.1, z;

Context	Example	Explanation
Expression with binary operator and operands of different numeric types	k+x; value 6.5	Value of int variable k is converted to type float before operation
Assignment statement with float target and int expression	z=k/m expression value is 1 value of z is 1.0	Expression is evaluated first. Result is converted to float for assignment
Assignment statement with int target and float expression	n=x*y expression value is 3.15 value of n is 3	Expression is evaluated first. Result is converted to int for assignment. Fraction is lost.

Basic Operations and Expressions

Explicit Conversions

- Why we need store data in different formats?
 - Data can be stored in the most compact form and converted when needed;
 - Disadvantage: Type mismatches can be missed by the compiler, it might possibly lead to losing information.
- Remembering the implicit conversion rules is one solution, but it's challenging to keep all the rules in mind. Are there **any alternative methods**?

Basic Operations and Expressions

Explicit Conversions

- You can force the use of a particular type and the implicit conversion can be ignored!!!
- Use a **cast operator** to convert the data type;
- Note that the **cast does not change a value stored** in a variable.
 - It only changes the type for the calculation.

`int` age;

age=1.2+5.978 -> automatically the result will be 7;

age=(`int`)1.2+(`int`)5.978=1+5=6 -> casts convert values first.

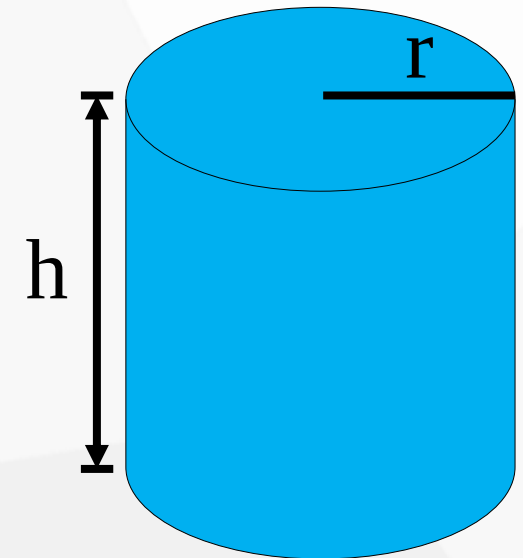
- ***Be noted**, it is not rounding to the nearest integer. **It is the floor operation.**

What is **function**?

- In mathematics, a function represents a mapping relationship from variables to a value. For example, the volume of a cylinder is determined by its height and radius. Thus, the mapping relationship can be expressed as:

$$v = f(h, r)$$

- Here, h and r are the arguments, and v is the result (**return value**).



$$\begin{aligned} v &= f(h, r) \\ &= \pi r^2 \times h \end{aligned}$$

What is function?

- In C programming, functions operate similarly to mathematical functions: they take variables as arguments and can return variables as results. However, it's crucial to specify the **data type** of each variable.

- Syntax:

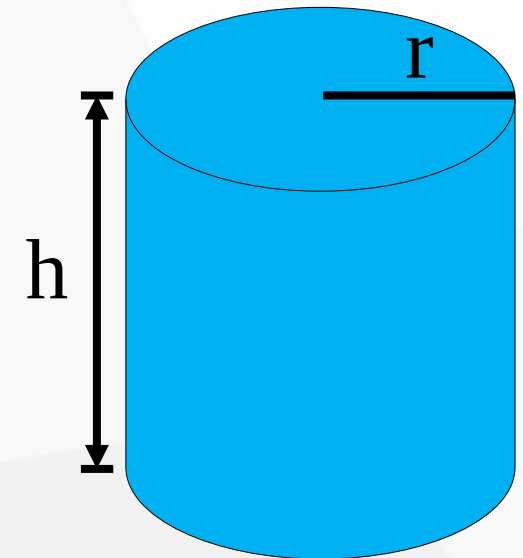
```
data_type func(data_type arg1, data_type arg2, ...)  
{  
    // function body  
    return results;  
}
```

What is function?

```
#include <stdio.h>
#define PI 3.1416

// Input args: @h: height, @r: radius
// Output result: volume.
float cylin_v(float h, float r) {
    float v = PI*r*r*h;
    return v;
}

int main()
{
    float h, r, v;
    printf("Cylinder's height and radius?\n");
    scanf("%f %f", &h, &r);
    v = cylin_v(h, r);
    printf("Its volume is %f\n", v);
    return 0;
}
```



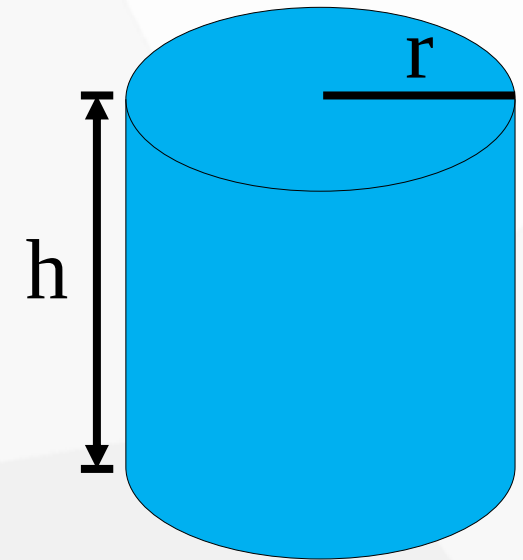
$$v = f(h, r) \\ = \pi r^2 \times h$$

What is function?

```
#include <stdio.h>
#define PI 3.1416

// Input args: @h: height, @r: radius
// Output result: volume.
float cylin_v(float h, float r) {
    float v = PI*r*r*h;
    return v;
}

int main()
{
    float h, r, v;
    printf("Cylinder's height and radius?\n");
    scanf("%f %f", &h, &r);
    v = cylin_v(h, r);
    printf("Its volume is %f\n", v);
    return 0;
}
```



$$v = f(h, r) \\ = \pi r^2 \times h$$

Thanks